

CONSTRAINT-BASED PROBLEM SOLVING
CSA1717 - Assessment Tool 2 – Artificial Intelligence

Name: Hemalakshmi H

Reg.No.: 192571049

BACKTRACKING SEARCH FOR DOCTOR-SHIFT ASSIGNMENT

1.1 Problem Definition

A hospital must assign three doctors (D1, D2, D3) to three shifts (Morning, Afternoon, Night) while satisfying the following constraints:

- D1 cannot work Night
- D3 cannot work Morning
- Only one doctor per shift
- D2 must work before D3 (Morning < Afternoon < Night)
- Each doctor must be assigned exactly one shift

This is a **constraint satisfaction problem (CSP)** solved using **Backtracking Search**.

1.2 Constraint Analysis

- **Unary Constraints:**
 - D1 ≠ Night
 - D3 ≠ Morning
- **Binary Constraints:**
 - D2 shift < D3 shift
- **Global Constraint:**
 - All shifts must be unique

These constraints restrict the domain of each doctor and guide the backtracking process.

1.3 Backtracking Search Process

Step 1: Assign D1 Domain(D1) = {Morning, Afternoon} Try D1 = Morning → Valid

Step 2: Assign D2 Domain(D2) = {Morning, Afternoon, Night}

- D2 = Morning → Conflict (Morning already taken)
- D2 = Afternoon → Valid (no conflict)

Step 3: Assign D3 Domain(D3) = {Afternoon, Night}

- D3 = Afternoon → Conflict (Afternoon taken by D2)
- D3 = Night → Valid
 - $D2 < D3 \rightarrow \text{Afternoon} < \text{Night}$
 - All shifts unique
 - $D3 \neq \text{Morning}$

1.4 Final Valid Schedule

- **D1 → Morning**
- **D2 → Afternoon**
- **D3 → Night**

This assignment satisfies all constraints without further backtracking.

BFS SEARCH IN A 5×5 GRID ENVIRONMENT

2.1 Problem Definition

A robot must move from Start (S) to Goal (G) in a 5×5 grid with obstacles. Movement allowed: Up, Down, Left, Right Cost of each move = 1 Heuristic: Manhattan Distance

2.2 Constraint Analysis

- Robot cannot move diagonally
- Robot cannot enter obstacle cells
- BFS ensures shortest path in terms of number of steps
- Manhattan Distance helps evaluate closeness to goal (though BFS does not use it for ordering)

2.3 BFS Search Process

Step 1: Initialization Queue = [S] Visited = {S}

Step 2: Expand S Add all valid neighbors (non-obstacle, inside grid) to queue.

Step 3: Level-by-Level Expansion FIFO order ensures shortest path exploration. Each expanded node generates neighbors that are added to the queue.

Step 4: Goal Detection When G is reached, BFS stops. Path is reconstructed using parent pointers.

2.4 Optimal Path and Cost

If BFS finds a path: $S \rightarrow n1 \rightarrow n2 \rightarrow \dots \rightarrow G$ and number of moves = k then **Optimal Cost** = $k \times 1 = k$

BFS guarantees the minimum-cost path because all moves have equal cost.

AUTONOMOUS RESCUE ROBOT – ONLINE SEARCH

3.1 Constraint Analysis

The robot operates in a **partially observable, dynamic environment** with:

- Movement allowed: Up, Down, Left, Right
- Some cells blocked (obstacles)
- Limited sensing (only adjacent cells visible)
- Normal cell cost = 1
- Risky zone cost = 3 (1 + 2 extra risk cost)
- Robot must minimize total cost
- Robot must update knowledge dynamically

Impact on Decision-Making

- Robot cannot plan full path initially
- Must update map after each move
- Must avoid risky zones unless necessary
- Must re-plan when new obstacles are discovered
- Must choose least-cost actions at every step

3.2 Solution Approach – Online Uniform Cost Search (UCS)

Steps Involved

1. **Initialize knowledge base** with only Start cell S.
2. **Sense adjacent cells** → classify as free, obstacle, or risky.
3. **Run UCS** to compute least-cost path using known information.
 - Priority queue ordered by cumulative cost
4. **Move to next lowest-cost cell**
5. **Sense again** from new position
6. **If new obstacles/risky zones discovered:**
 - Update map
 - Re-run UCS from current position
7. Continue until Goal (G) is reached.

This makes the robot an **online search agent**.

3.3 Application of AI Concepts

Intelligent Agent

Robot behaves as an intelligent agent because it:

- Perceives environment
- Updates internal model
- Chooses cost-minimizing actions
- Acts autonomously

Rationality

Robot is rational because it:

- Avoids obstacles
- Minimizes movement + risk cost
- Re-plans when new information appears
- Always selects least-cost path

Environment Type

- **Partially observable** – only adjacent cells visible
- **Dynamic** – new hazards discovered during movement
- **Sequential** – each action affects future decisions
- **Unknown** – full map not available initially

3.4 Justification of Chosen Approach

Uniform Cost Search with online re-planning is ideal because:

- Handles **variable costs** (normal vs risky zones)
- Guarantees **least-cost path**

- Works well in **partially observable** environments
- Supports **dynamic updates** when new obstacles appear
- Safer and more optimal than BFS/DFS

Thus, UCS satisfies all constraints: safety, cost minimization, adaptability, and intelligent decision-making.